School Bus Tracking SaaS — Product Requirements Document (PRD)

Product Name: BusTrackr (working title, change as you like) **Version:** 1.0 — Pilot Release **Goal:** Deploy to one real school, validate, then scale. **Platform:** Web (Admin Panel) + iOS + Android (Driver App + Parent App) **Model:** Free for pilot, SaaS subscription after validation

---

Product Vision

A school bus tracking system that gives parents real-time peace of mind, gives school admins full fleet control, and gives drivers a simple tool that works even in poor network conditions. Built the way Uber and Zomato build tracking — event-driven, offline-resilient, sub-500ms updates.

Three Apps, One System

App	Who Uses It	Platform
Admin Panel	School admin / principal	Web (Next.js)
Driver App	Bus driver	iOS + Android (React Native)
Parent App	Parent / guardian	iOS + Android (React Native)

Roles and Permissions Summary

Action	Admin	Driver	Parent
Register school	Yes	No	No
Create routes/stops	Yes	No	No
Create driver accounts	Yes	No	No
Create student accounts	Yes (also parent)	No	Yes (own child)
Start/end trip	No	Yes	No
Send GPS pings	No	Yes	No
View live fleet map	Yes (all buses)	No	No
Track own child's bus	No	No	Yes
View attendance	Yes (all)	Yes (own trip)	Yes (own child)
Configure alerts	Yes	No	Yes (own)
View analytics	Yes	No	No
Trigger SOS	No	Yes	No

Module List Overview

#	Module	App	Priority
M1	Authentication	All 3	P0
M2	School Setup	Admin	P0
M3	Route and Stop Management	Admin	P0
M4	Driver Management	Admin	P0
M5	Student Management	Admin	P0
M6	Bus Management	Admin	P0
M7	Admin Dashboard	Admin	P0
M8	Live Fleet Map	Admin	P0
M9	Trip Monitoring (Admin)	Admin	P0
M10	Alerts and Violations	Admin	P1
M11	Trip History and Replay	Admin	P1
M12	Analytics	Admin	P1
M13	Driver App — Device Precheck	Driver	P0
M14	Driver App — Duty Start and Trip	Driver	P0
M15	Driver App — Live Dashboard	Driver	P0
M16	Driver App — Stop Arrival and Attendance	Driver	P0
M17	Driver App — SOS and Alerts	Driver	P0
M18	Driver App — End Trip and Summary	Driver	P0
M19	Driver App — Offline Resilience	Driver	P0
M20	Driver App — Settings	Driver	P1
M21	Parent App — Signup and Child Setup	Parent	P0
M22	Parent App — Dashboard	Parent	P0
M23	Parent App — Live Tracking	Parent	P0
M24	Parent App — Notifications	Parent	P0
M25	Parent App — Trip History	Parent	P1
M26	Parent App — Settings	Parent	P1
M27	Backend — GPS Pipeline	Backend	P0
M28	Backend — Geofence Engine	Backend	P0
M29	Backend — Notification Engine	Backend	P0
M30	Backend — Offline Sync	Backend	P0

MODULE DETAILS AND TASKS

M1 — Authentication (All Apps)

Purpose: Secure login and registration for all three user types using JWT tokens. OTP-based password recovery. Each user is always scoped to their school — no cross-school data is ever accessible.

User Stories:

- As a school admin, I want to register my school and create an account so I can start setting up the system.
- As a driver, I want to log in with my phone number and password so I can start my duty.
- As a parent, I want to sign up, add my children, and log in easily.

Acceptance Criteria:

- JWT token issued on login, expires in 24h, refresh token valid 30 days.
- All tokens contain school_id so every API call is auto-scoped.
- OTP sent via SMS (use Twilio free trial or Fast2SMS for India).
- Wrong password 5 times = account locked for 15 minutes.
- Passwords hashed with bcrypt, never stored plain.

Tasks — M1

T1.1 — Set up Node.js auth service with Express and JWT library. Prompt hint: "Create a Node.js Express auth service with JWT login, bcrypt password hashing, and refresh token support. User model has id, school_id, role (admin/driver/parent), phone, email, password_hash, is_active."

T1.2 — Build POST /auth/register-school endpoint. Accepts school name, phone, email, password, lat, lng, logo_url. Creates school record and admin user in one transaction.

T1.3 — Build POST /auth/login endpoint. Accepts phone or email + password. Returns access_token and refresh_token. Embeds user_id, school_id, role in JWT payload.

T1.4 — Build POST /auth/refresh endpoint. Accepts refresh_token. Returns new access_token. Validates refresh token is not expired or revoked.

T1.5 — Build POST /auth/otp/send and POST /auth/otp/verify endpoints. Send 6-digit OTP to phone. Store OTP in Redis with 5 minute TTL. Verify OTP and issue password reset token.

T1.6 — Build auth middleware. Express middleware that reads Authorization header, verifies JWT, attaches req.user = { id, school_id, role } to every protected request.

T1.7 — Build admin login page (Next.js). Simple form: email/phone + password. Show error states. On success redirect to dashboard.

T1.8 — Build admin forgot password flow (Next.js). Enter phone → receive OTP → enter OTP + new password → redirect to login.

T1.9 — Build driver login screen (React Native). Phone number input + password input. Show loading state. Store token in SecureStore. On success go to Device Precheck screen.

T1.10 — Build parent signup screen (React Native). Step 1: Parent details (name, phone, email, password, alternate number). Step 2: Add children (name, class, gender, school, route, stop). Can add multiple children. Submit creates account.

T1.11 — Build parent login screen (React Native). Phone/email + password. Forgot password OTP flow same as admin.

M2 — School Setup (Admin Panel)

Purpose: When a school admin registers, they configure their school profile including location coordinates which become the fixed center point for all maps and routes.

User Stories:

- As an admin, I want to set my school's exact location so all routes connect to it correctly.
- As an admin, I want to upload my school logo and update contact details.

Acceptance Criteria:

- School location (lat/lng) is set once at registration. Can be updated only by admin.
- School location is the mandatory start/end point for all routes.
- Logo stored in Cloudflare R2 / Supabase storage.

Tasks — M2

T2.1 — Build school registration page with map picker (Next.js). Show a Mapbox or Leaflet map. Admin can click to pin school location OR type lat/lng manually. Store coordinates.

T2.2 — Build school profile settings page (Next.js). Edit school name, phone, email, logo upload. Show current coordinates with option to update.

T2.3 — Build PATCH /schools/:id endpoint. Updates school profile. Only admin of that school can call it. Validate school_id from JWT matches.

T2.4 — File upload endpoint for school logo. POST /uploads/logo. Accepts image, stores to R2/Supabase storage, returns URL.

M3 — Route and Stop Management (Admin Panel)

Purpose: Admin creates routes, each connecting multiple stops to the school. Every route has a morning version (stops → school) and evening version (school → stops, auto-reversed). Stops are plotted on a map or entered as coordinates.

User Stories:

- As an admin, I want to create a route with multiple stops so a driver knows where to go.
- As an admin, I want to edit or delete stops without affecting active trips.
- As an admin, I want changes to sync instantly to the driver app and parent app.

Acceptance Criteria:

- A route must have a name, at least 1 stop, and school as the terminal point.
- Stop has name, lat, lng, sequence order, and geofence radius (default 60m, configurable 30–90m).
- Morning trip sequence: stop 1 → stop 2 → ... → school. Evening trip is exact reverse.
- Any route update triggers a WebSocket broadcast to all connected driver/parent clients on that route.
- Route cannot be deleted if a trip is currently active on it.

Tasks — M3

T3.1 — Build routes list page (Next.js). Table of all routes with name, number of stops, assigned drivers, status (active/inactive). Add route button.

T3.2 — Build create/edit route page with interactive map (Next.js). Use Mapbox GL JS or Leaflet. Show school as a fixed marker. Admin clicks on map to add stops. Drag stops to reorder. Each stop shows name input and radius slider. Show animated route line connecting all stops to school.

T3.3 — Build POST /routes endpoint. Create route with stops array. Each stop gets sequence_order assigned. Auto-calculate estimated distance using OSRM routing between stops.

T3.4 — Build PATCH /routes/:id endpoint. Update route name or description. Broadcast update event on Redis channel route:{id}:updated.

T3.5 — Build POST /routes/:id/stops, PATCH /stops/:id, DELETE /stops/:id endpoints. Add, edit, delete stops. On any change broadcast sync event.

T3.6 — Build real-time route sync. When admin saves any route change, emit WebSocket event to all driver and parent clients subscribed to that route. They should reload route data without needing to refresh.

T3.7 — Build route preview on admin panel. After creating a route, show a preview: animated route line on map, stop list with sequence, estimated total distance and time.

M4 — Driver Management (Admin Panel)

Purpose: Admin creates driver accounts. Drivers cannot self-register. Admin can view driver attendance, duty history, and suspend or remove drivers.

User Stories:

- As an admin, I want to register a new driver so they can log into the driver app.
- As an admin, I want to see which drivers checked in today and view their duty history.

Acceptance Criteria:

- Driver account created by admin only. Initial password set by admin, driver changes on first login.
- Driver profile has: name, phone, photo, licence number, experience years, age.
- Admin can mark driver as inactive (they lose app access immediately).
- Driver attendance log auto-generated from device precheck timestamps.

Tasks — M4

T4.1 — Build drivers list page (Admin Panel). Table: name, phone, licence, status (active/on-duty/inactive), today's check-in time. Filter by status.

T4.2 — Build create/edit driver form (Admin Panel). Fields: name, phone (becomes login), password, photo upload, licence number, experience, age. Submit creates user record with role=driver.

T4.3 — Build POST /users (driver creation) endpoint. Admin creates driver. school_id from JWT is auto-attached. Returns driver ID and login credentials.

T4.4 — Build driver attendance view (Admin Panel). Calendar-style view showing which days driver did duty, check-in time, check-out time, total hours. Exportable as CSV.

T4.5 — Build driver detail page. Show driver info, all past trips (with date, route, distance, alerts triggered), attendance summary.

M5 — Student Management (Admin Panel)

Purpose: Students are linked to exactly one route and one stop. Admin can create students directly or parents can create them during signup (two-way creation). Attendance is auto-generated from onboard/drop events during trips.

User Stories:

- As an admin, I want to create a student record and link it to a route and stop.
- As an admin, I want to bulk import students from a CSV file.

- As an admin, I want to view attendance for any student for any date range.
- As a parent, I want to create my child's profile during app signup.

Acceptance Criteria:

- Student has: name, class, section, gender, photo, parent_id, route_id, stop_id.
- Student is always linked to exactly one stop (their boarding/alighting point).
- CSV import supports columns: name, class, section, gender, parent_phone, route_name, stop_name.
- Attendance report exportable per student, per route, per date range as CSV.
- When admin creates student, parent can log in and see the child automatically.

Tasks — M5

T5.1 — Build students list page (Admin Panel). Table: name, class, route, stop, parent name, attendance this month. Search and filter by route, class, section.

T5.2 — Build create/edit student form (Admin Panel). Fields: name, class, section, gender, photo, parent phone (auto-links or creates parent), route selector, stop selector (filtered by route).

T5.3 — Build POST /students endpoint. Create student. Validate route_id and stop_id belong to same school. Link parent by phone number (create parent user if not exists).

T5.4 — Build CSV bulk import for students. POST /students/import accepts CSV file. Parse, validate each row, create students in bulk. Return success/failure summary with row numbers for failed rows.

T5.5 — Build attendance report page (Admin Panel). Select student or route + date range. Show calendar grid of onboard/drop status per day. Export as CSV.

T5.6 — Build GET /attendance endpoint. Query params: student_id or route_id, date_from, date_to. Returns attendance records with onboard/drop timestamps.

M6 — Bus Management (Admin Panel)

Purpose: Admin registers each bus in the fleet. Buses are assigned to routes. A bus can only be on one active trip at a time.

Acceptance Criteria:

- Bus has: plate number, name/number, capacity, school_id, is_active.
- A bus cannot start a new trip if it already has an active trip.
- Admin can deactivate a bus (it will not appear in driver's bus selection).

Tasks — M6

T6.1 — Build buses list page (Admin Panel). Table: bus name, plate number, capacity, current status (available/on-trip/inactive), current driver if on trip.

T6.2 — Build create/edit bus form. Fields: name/number, plate number, capacity. Submit calls POST /buses.

T6.3 — Build POST /buses, PATCH /buses/:id, DELETE /buses/:id endpoints. Standard CRUD. Delete only allowed if no active trips. Deactivate sets is_active=false.

M7 — Admin Dashboard (Admin Panel)

Purpose: The first screen after login. Gives a full operational view of the school's fleet right now and today's summary stats.

Key Metrics to Show:

- Total buses / Active buses right now
- Total trips today / Active trips / Completed trips
- Driver attendance today (checked in / total drivers)
- Student attendance today (onboarded / total students)
- Active routes / Total routes
- Any open alerts (speeding, SOS, breakdown)

Acceptance Criteria:

- All numbers update in real time via WebSocket when trips are active.
- Alert count badge shows red if any unresolved alerts.
- Quick links to active trip detail, driver profile, route map.

Tasks — M7

T7.1 — Build dashboard layout (Next.js). KPI cards row at top. Alert banner if active alerts. Below: active trips table + driver attendance table side by side.

T7.2 — Build GET /dashboard/stats endpoint. Returns: bus counts, trip counts, driver attendance count, student attendance count, alert count. All scoped to school_id from JWT.

T7.3 — Wire dashboard to WebSocket. On trip:started, trip:ended, alert:created events, refresh the relevant KPI cards without full page reload.

T7.4 — Build active trips table on dashboard. Columns: route name, driver name, bus, students onboard, current speed, last GPS ping time. Row click goes to live trip view.

M8 — Live Fleet Map (Admin Panel)

Purpose: A full-screen map centered on the school showing all buses currently on active trips as animated moving markers. Also includes a terminal-style sidebar showing all stops for active routes with live status.

Acceptance Criteria:

- School is a fixed pin at center, never moves.
- Each bus is a distinct colored animated marker. Marker smoothly interpolates between GPS points (not jump-cuts).
- Clicking a bus marker opens a sidebar with: driver name, route name, speed, next stop, ETA to school, students onboard count.
- Terminal-style view: vertical list of all stops for each active route with status dot (waiting / bus approaching / bus passed).
- Map auto-fits to show all active buses without zooming out too far.
- If no active trips, show empty state with all routes drawn as faint lines.

Tasks — M8

T8.1 — Build live fleet map page (Next.js + Mapbox GL JS). Initialize map centered on school coordinates. Load school pin. Load route polylines as faint background lines.

T8.2 — Build animated bus markers. Each bus is a custom marker (bus icon SVG). On each GPS update, smoothly animate the marker to new position using linear interpolation over 1 second. Rotate marker based on heading.

T8.3 — Subscribe to fleet WebSocket channel. Connect to ws: school:{id}:fleet. On each location event, find the bus marker and move it. If bus marker does not exist yet, create it.

T8.4 — Build bus info popup. Click on bus marker shows panel: driver name, route, speed, next stop name + distance, ETA to school, onboard student count, last ping time.

T8.5 — Build terminal-style stop status sidebar. For each active route, show vertical list of stops. Each stop has a colored status indicator: gray (not yet reached), amber (bus approaching, within 500m), green (passed). Update in real time.

M9 — Trip Monitoring (Admin Panel)

Purpose: Admin can click into any active trip and see full detail: live map of that single bus, student attendance list for that trip, telemetry, and any alerts.

Acceptance Criteria:

- Shows live GPS trace (path the bus has taken so far) as a drawn line on map.
- Student list shows: name, stop, status (waiting / onboarded / dropped).
- Telemetry bar: current speed, total distance covered, trip start time, estimated end time.
- Alert log for that trip shown in real time.

Tasks — M9

T9.1 — Build trip detail page (Next.js). URL: /trips/[tripId]. Load trip data. Show single bus on map with route line. Show sidebar with student attendance.

T9.2 — Build live GPS trace. As location events come in via WebSocket, append to a polyline drawn on the map. Show the path the bus has taken so far in a different color from the planned route.

T9.3 — Build live student attendance panel. List of all students for that route. Each row shows name, stop, current status badge. Update in real time when driver marks onboard/drop.

T9.4 — Build GET /trips/:id/detail endpoint. Returns trip info, full student list with attendance status, telemetry summary, alert list.

M10 — Alerts and Violations (Admin Panel)

Purpose: System auto-generates alerts for speeding, SOS/breakdown, route deviation, bus idle too long. Admin can view, acknowledge, and resolve alerts.

Alert Types:

- Speeding — bus exceeded configured speed threshold (default 40 km/h in school zone, 60 km/h on highway, configurable)
- SOS / Breakdown — driver triggered panic button
- Idle / Halt — bus stopped for more than configured time (default 5 minutes) mid-route
- Off-route deviation — bus moved more than 200m from planned route

Acceptance Criteria:

- Each alert has: type, trip_id, driver, lat/lng at trigger, speed at trigger, timestamp.
- Speeding alert auto-triggers in Location Service when speed > threshold.
- SOS alert pushes notification to admin immediately via FCM.
- Admin can mark alert as resolved with a note.
- Alert history exportable as CSV.

Tasks — M10

T10.1 — Build alerts list page (Admin Panel). Filter by type, date range, driver, route. Each row: type badge, driver, route, time, speed (if speeding), GPS location link, resolved status.

T10.2 — Build speed threshold configuration. Settings page where admin sets speed threshold per route or globally. Stored in school settings table.

T10.3 — Build alert detail page. Show alert info, map pin at alert location, driver info, trip context, resolution log.

T10.4 — Build POST /alerts/sos endpoint. Driver triggers SOS. Store alert. Immediately publish to Redis. Notification service picks up and sends FCM to admin + all parents on that route.

T10.5 — Build auto-speeding alert in Location Service (Go). On each GPS ping, compare speed_kmh to school's configured threshold. If exceeded and last speeding alert for this trip was more than 60 seconds ago, create alert record and publish event.

T10.6 — Build PATCH /alerts/:id/resolve endpoint. Admin resolves alert with optional note. Updates resolved_at and resolution_note.

M11 — Trip History and Replay (Admin Panel)

Purpose: Admin can select any past trip, see a full summary, and replay the GPS route on a map with a timeline scrubber.

Acceptance Criteria:

- Trip list filterable by route, driver, date, bus.
- Trip detail shows: total distance, duration, average speed, delays, student attendance summary, alerts.
- Route replay: animated bus marker moves along the recorded GPS path. User can scrub timeline.

Tasks — M11

T11.1 — Build trip history list page (Admin Panel). Table with filters. Each row shows date, route, driver, bus, duration, distance, student count, alert count.

T11.2 — Build trip detail/summary page. Show summary cards + map with full GPS trace drawn. Student attendance table (who was onboard, who was absent).

T11.3 — Build GPS replay feature. Load all GPS pings for the trip from TimescaleDB ordered by timestamp. Animate bus marker along the path. Play/pause button. Speed slider (1x, 2x, 5x). Timeline scrubber.

T11.4 — Build GET /trips/:id/gps-log endpoint. Returns all GPS pings for a trip ordered by time. Used for replay.

M12 — Analytics (Admin Panel)

Purpose: Give admin useful insights about fleet performance over time.

Key Reports:

- Average delay per route (planned vs actual arrival time at school)
- Speeding incidents per driver per month
- Monthly total distance per bus
- Student attendance rate per route
- Driver duty hours per month

Acceptance Criteria:

- Date range picker for all reports.
- Charts rendered as bar/line charts (use Recharts or Chart.js).
- All reports exportable as CSV.

Tasks — M12

T12.1 — Build analytics page layout (Next.js). Tabs: Route performance / Driver performance / Student attendance / Fleet usage. Date range filter at top.

T12.2 — Build GET /analytics/delays endpoint (Python FastAPI). Calculates average delay per route by comparing trip end time to historical baseline. Returns per-route stats.

T12.3 — Build GET /analytics/speeding endpoint. Returns speeding count per driver for date range.

T12.4 — Build GET /analytics/distance endpoint. Returns total distance per bus per month from trips table.

T12.5 — Build GET /analytics/attendance endpoint. Returns attendance rate (onboarded / enrolled) per route per day.

T12.6 — Render charts in admin panel. Use Recharts. Bar chart for delays, line chart for distance over time, pie chart for attendance rate.

M13 — Driver App — Device Precheck

Purpose: Every time driver opens the app and starts duty, the app checks that all required sensors and permissions are working. Results are logged to server.

Checks Required:

- GPS enabled and getting signal (accuracy < 20m)
- Location permission granted (precise, always)
- Internet connection (show type: WiFi / 4G / 3G / 2G)
- Battery level (warn if below 20%)
- Background location permission enabled

Acceptance Criteria:

- Each check shows as green tick or red cross with a label.
- Driver cannot proceed to duty start if GPS or location permission is red.
- Battery warning is shown but does not block.
- Precheck results (with timestamp) posted to server as device_precheck log.

Tasks — M13

T13.1 — Build precheck screen (React Native). List of checks with animated status indicators. Check all on screen mount. Show Retry button if any fail.

T13.2 — GPS accuracy check. Use expo-location getCurrentPositionAsync with high accuracy. If accuracy > 20m show warning. If no signal after 10s show error.

T13.3 — Permission checks. Use expo-location requestForegroundPermissionsAsync and requestBackgroundPermissionsAsync. Show red if denied with deep link to phone settings.

T13.4 — Battery check. Use expo-battery. Show level and warn if < 20%. Does not block duty start.

T13.5 — Network check. Use expo-network. Show connection type. Warn if offline but do not block (offline mode will handle it).

T13.6 — Post precheck log to server. POST /driver-duty/precheck with all check results, device info, timestamp. Store in driver_duty table as jsonb.

M14 — Driver App — Duty Start and Trip

Purpose: Driver selects which bus they are taking and which route, then picks morning or evening shift. System creates a trip record and opens the live dashboard.

Acceptance Criteria:

- Driver can only select buses and routes assigned by admin and currently available (not on another active trip).
- Morning shift: route goes stop-by-stop to school. Evening shift: starts at school, goes stop-by-stop to drop students.
- Trip record created in DB with status=active when driver confirms start.
- Route map and student list loaded into app from server before trip starts.
- If driver has no assigned route, show error and contact admin message.

Tasks — M14

T14.1 — Build duty start screen (React Native). Step 1: Select bus (dropdown, shows plate + name). Step 2: Select route (list, shows stops count + estimated time). Step 3: Select shift (morning / evening toggle). Confirm button. Show loading while trip is created.

T14.2 — Build GET /driver/available-buses endpoint. Returns buses for driver's school that are not currently on an active trip.

T14.3 — Build GET /driver/routes endpoint. Returns routes available for this driver (all school routes or specifically assigned ones).

T14.4 — Build POST /trips/start endpoint. Creates trip record with school_id, route_id, bus_id, driver_id, shift, status=active, start_time=now. Returns full trip object with route stops and student list embedded.

T14.5 — Cache route and student data locally on trip start. On trip created, store full route JSON (stops with lat/lng, student list per stop) into AsyncStorage. This ensures stop info and student names are available offline.

M15 — Driver App — Live Dashboard

Purpose: The main screen while a trip is in progress. Shows animated map with route and stops, telemetry bar, and easy access to student lists.

Screen Layout:

- Top bar: route name, shift, trip timer
- Telemetry bar (always visible): current speed / next stop name / distance to next stop / ETA to next stop / total distance remaining
- Main: full-screen animated map with route line, stop markers, current bus position
- Bottom drawer: stop list (scrollable), each stop shows stop name, number of students, status (upcoming / current / done)
- Clicking any stop opens that stop's student list

Acceptance Criteria:

- Bus position on map updates every 1 second smoothly (no jump-cuts, interpolated movement).
- ETA recalculates every 10 seconds using OSRM with current position.
- Stop status updates automatically as stops are completed.
- Map auto-centers on bus with appropriate zoom.

Tasks — M15

T15.1 — Build live dashboard screen layout (React Native). Use react-native-maps with Mapbox or Google Maps provider. Set up top bar, telemetry bar component, map view, bottom drawer.

T15.2 — Build GPS streaming. Use expo-location watchPositionAsync with 1 second interval. On each location update: update local state for telemetry display, store in IndexedDB queue, send via WebSocket.

T15.3 — Build WebSocket connection manager. Connect to ws://api/trip:{tripId}. Send GPS ping every second. Handle disconnect with exponential backoff reconnect. Show connection status indicator dot.

T15.4 — Build telemetry bar. Real-time display: speed (from GPS), distance to next stop (calculated from current lat/lng to next stop lat/lng using Haversine formula), ETA (from OSRM or estimated from speed + distance).

T15.5 — Build animated bus marker on map. Smooth interpolation between GPS positions. Rotate marker to heading direction. Marker stays centered with slight offset as map follows.

T15.6 — Build stop list bottom drawer. Scrollable list of stops in trip order. Each row: stop name, student count for that stop, status badge (upcoming/active/done). Tap row opens student list for that stop.

T15.7 — Build ETA recalculation. Every 10 seconds call GET /route/eta with current lat/lng and remaining stops. Update telemetry bar with new ETAs.

M16 — Driver App — Stop Arrival and Attendance

Purpose: When the bus enters the geofence radius of a stop, the student list for that stop pops up automatically. Driver marks students onboard with a single tap, or marks absent with a double tap. When the bus moves away, the list closes. At school (end of morning trip), driver can tap "Drop all" to mark everyone as dropped.

Morning Trip Logic:

- At each stop: student list shows all enrolled students for that stop → driver taps to mark onboard → double tap to mark absent
- At school: student list shows all onboarded students → driver taps "Drop all" or marks individually

Evening Trip Logic:

- At school: student list shows all students enrolled on this route → driver taps to mark who is getting on the bus
- At each stop: student list shows students for that stop who are onboard → driver taps to mark as dropped

Acceptance Criteria:

- Auto-open is triggered when bus enters stop's configured radius (30–90m, set by admin).
- Auto-close happens when bus moves more than 150m from stop or bus speed exceeds 10 km/h.
- Single tap = onboarded. Double tap = absent / not onboarded. Tap again to undo.
- "Onboard all" and "Drop all" single-tap bulk actions available.

- Each onboard/drop event sent to server immediately (or queued if offline).
- Parent gets push notification when their child is onboarded or dropped.

Tasks — M16

T16.1 — Build stop arrival detection. In the GPS update loop, for each incoming position check distance to next expected stop using Haversine. If distance < stop.radius_meters, trigger stop arrival event.

T16.2 — Build auto-open student list. On stop arrival event, show bottom sheet with students for that stop. Animate in from bottom.

T16.3 — Build student list UI with tap/double-tap. Each student row: photo thumbnail, name, class. Single tap: highlight green (onboard). Double tap: highlight red (absent). "Onboard all" and "Skip all" buttons at bottom.

T16.4 — Build auto-close logic. Watch bus speed and distance from current stop. If speed > 10 km/h or distance > 150m from stop, dismiss student list.

T16.5 — Build POST /attendance/onboard endpoint. Body: trip_id, student_id, stop_id, lat, lng, timestamp. Creates attendance record with status=onboard. Triggers parent notification.

T16.6 — Build POST /attendance/drop endpoint. Body: trip_id, student_id, stop_id, lat, lng, timestamp. Updates attendance record status to dropped. Triggers parent drop notification.

T16.7 — Build offline attendance queue. If WebSocket is down, store attendance events in IndexedDB with type (onboard/drop), student_id, timestamp. Flush when connection restores.

T16.8 — Build "Drop all" bulk action. Single button at school arrival. Calls POST /attendance/drop-all with trip_id. Server marks all onboarded students as dropped and sends notifications to all parents.

M17 — Driver App — SOS and Alerts

Purpose: Driver can send a panic/SOS/breakdown alert to admin and all parents on the route with a single button press.

Acceptance Criteria:

- SOS button visible at all times during active trip (not easily accidentally triggered — require hold for 2 seconds or confirmation dialog).
- Alert sent with current GPS location, timestamp, trip_id.
- Admin receives FCM push notification immediately.
- All parents on that route receive FCM push notification.
- Alert appears in admin alert log.

Tasks — M17

T17.1 — Build SOS button UI (React Native). Red button in corner of dashboard. Tap shows confirmation dialog: "Send SOS alert to admin and parents?" with Cancel / Send buttons.

T17.2 — Wire SOS to POST /alerts/sos. On confirm, call endpoint with current lat/lng, trip_id, timestamp. Show confirmation toast "Alert sent".

T17.3 — Show sent SOS status in driver dashboard. After sending, show a banner "SOS sent — admin notified" until driver dismisses it.

M18 — Driver App — End Trip and Summary

Purpose: When all students are dropped, driver taps "End Trip". System shows a summary of the trip.

Summary Shows:

- Total distance traveled
- Trip duration (start to end time)
- Number of students onboarded / dropped / absent
- Average speed
- Number of stops completed
- Any alerts triggered during trip

Acceptance Criteria:

- End trip only available after all students on the route have a drop record (or are marked absent). If not, show warning but allow override.
- Trip summary saved to server. Status updated to completed.
- Parents of dropped students receive "Your child has been dropped" notification if not already sent.
- Driver can view this summary later in trip history.

Tasks — M18

T18.1 — Build end trip confirmation screen (React Native). Show warning if any students not yet dropped. Confirm button calls POST /trips/:id/end.

T18.2 — Build POST /trips/:id/end endpoint. Set status=completed, end_time=now. Calculate total_distance_km from GPS pings. Calculate avg_speed. Return full summary object.

T18.3 — Build trip summary screen (React Native). Display all summary stats in cards. Show route map with full GPS trace. Button to go back to duty start.

M19 — Driver App — Offline Resilience

Purpose: The app must continue recording GPS and attendance data even when there is no internet. Data syncs automatically when connection returns. This was the core failure in your previous app.

Offline Capabilities:

- GPS continues recording to local queue (IndexedDB / AsyncStorage)
- Attendance taps stored locally
- Route and student data available from cache (loaded at trip start)
- Connection status indicator always visible
- On reconnect, all queued data batch-posted to server in order

Acceptance Criteria:

- App never crashes or shows error screen due to network loss.
- Connection indicator: green dot (WS live), yellow dot (HTTP fallback), red dot (fully offline, buffering).
- On reconnect, batch sync happens automatically in background without interrupting driver.
- All GPS pings and attendance events that were buffered get synced correctly with their original timestamps.
- If app is closed and reopened mid-trip (e.g. app crash), it restores the in-progress trip from AsyncStorage.

Tasks — M19

T19.1 — Build local GPS queue. On every GPS update, write to AsyncStorage queue BEFORE attempting to send. On successful WS send, remove from queue.

T19.2 — Build connection state manager. Monitor WebSocket state and NetInfo. States: ONLINE_WS, ONLINE_HTTP, OFFLINE. Show indicator dot in all app screens.

T19.3 — Build batch sync on reconnect. When transitioning from offline to online, read all queued GPS pings and attendance events from AsyncStorage, call POST /location/batch and POST /attendance/batch in order, clear queue on success.

T19.4 — Build trip state persistence. When trip is started, write trip state (trip_id, route, shift, current stop index, student attendance) to AsyncStorage. On app restart, check if there is an in-progress trip and restore to live dashboard.

T19.5 — Build POST /location/batch endpoint. Accepts array of GPS pings with timestamps. Inserts in chronological order into TimescaleDB. Re-runs geofence checks on each point to trigger any missed stop arrivals.

M20 — Driver App — Settings

Purpose: Language selection, permissions management, voice assistance test, app sharing.

Tasks — M20

T20.1 — Build settings screen (React Native). Language selector (Telugu / Hindi / English). Uses i18next for translations. All app strings translated.

T20.2 — Permissions management section. Show current state of each permission. Deep link to phone settings if user needs to change them.

T20.3 — Voice assistance test. Button to test text-to-speech: says next stop name aloud. Uses expo-speech. Useful for drivers to hear "Approaching Dilsukhnagar stop" without looking at phone.

T20.4 — Share app link. Share button generates Play Store / App Store link. Uses React Native Share.

M21 — Parent App — Signup and Child Setup

Purpose: Parent creates account, adds one or more children, selects each child's route and stop. Can also use credentials given by school admin to log in directly.

Acceptance Criteria:

- Parent can add multiple children but each child has exactly one route and one stop.
- If admin already created the student, parent logs in and child appears automatically (linked via parent phone number).
- Route and stop selection shows only the school's available options.

Tasks — M21

T21.1 — Build parent signup screen (React Native). Two-step: Step 1: Parent details (name, phone, email, password, alternate phone). Step 2: Add child (name, class, gender, photo, select bus/route/stop). Add another child button. Submit all.

T21.2 — Build child-route-stop selector. After entering child details, show route list (fetched from server). On route select, show stops on that route. Parent selects their nearest stop.

T21.3 — Build POST /parents/register endpoint. Creates parent user and student(s) in one transaction. Links to school by matching route/stop to school_id.

T21.4 — Build add/edit child screen (React Native). Accessible from settings after signup. Edit child's name, class, gender, route, stop.

M22 — Parent App — Dashboard

Purpose: The home screen for parents. Shows everything needed at a glance about their child's current trip status.

Displays:

- Child's name and current status badge (On bus / Dropped / No trip today / Waiting)
- Next stop name and ETA to next stop
- Current bus speed
- Distance from bus to their stop
- ETA to their stop specifically
- Any delay indicator
- Quick access to live map

No Trip State:

- If no active trip on child's route: show last trip summary and tomorrow's expected trip time.

Acceptance Criteria:

- All values update via WebSocket when trip is active.
- If multiple children: tabs or a switcher at top to flip between children.
- Tapping ETA card opens full live map.

Tasks — M22

T22.1 — Build parent dashboard screen (React Native). Child status card at top with status badge. Metrics row (speed, ETA, distance). Map preview thumbnail that expands to full map. Last notification section.

T22.2 — Subscribe to child's trip WebSocket. On dashboard mount, find child's active trip and connect to ws: trip:{tripId}. On each location event, update all dashboard metrics.

T22.3 — Build ETA-to-stop calculation. Using current bus position and child's stop coordinates, calculate distance and ETA continuously. Display with countdown feel.

T22.4 — Build no-trip empty state. If no active trip: show last trip date, child's status from last trip. Show expected schedule if available.

M23 — Parent App — Live Tracking

Purpose: Full-screen animated map showing the bus moving in real time on the route. Shows route line, all stops, and the bus marker.

Acceptance Criteria:

- Bus marker moves smoothly with interpolation.

- Child's stop highlighted with a different color marker.
- Route line shown with stops as dots.
- Estimated arrival time to child's stop shown as a pill overlay on the map.
- If bus has already passed child's stop, show that clearly.

Tasks — M23

T23.1 — Build live map screen (React Native + react-native-maps). Full screen map. Load route polyline. Show all stops as small dots. Highlight parent's stop as a larger colored dot.

T23.2 — Animate bus marker. Same interpolation logic as admin panel. Smooth movement, rotating marker based on heading.

T23.3 — Build ETA overlay. Floating pill at bottom showing "Reaching your stop in 8 min". Updates every 10 seconds.

T23.4 — Build stop-passed indicator. If bus has moved past parent's stop without dropping child, show alert: "Bus may have passed your stop".

M24 — Parent App — Notifications

Purpose: Parents receive push notifications at key moments during the trip. They can configure when and how they are notified.

Notification Types:

- Trip started: "Your child's bus has started the route"
- Bus approaching (configurable): 2 stops before their stop OR when ETA ≤ configured minutes
- Child onboarded: "Aarav has boarded the bus at Dilsukhnagar stop"
- Child dropped: "Aarav has been dropped at Madhapur stop"
- SOS/breakdown: "Bus breakdown alert on your child's route"
- Evening pre-alert: "Your child's bus will arrive in approximately X minutes"

Acceptance Criteria:

- Notification preferences saved per child.
- Parent can choose: notify by stops count (1 stop / 2 stops / 3 stops before) OR by time (5 min / 10 min / 15 min before).
- Ringer-style alert (plays sound even if phone on silent) for approaching and drop notifications — uses high-priority FCM channel.
- All notifications logged and viewable in notification history screen.

Tasks — M24

T24.1 — Build notification preferences screen (React Native). Per child settings: arrival alert type (stops / time), how many stops or minutes before, sound/vibration toggle.

T24.2 — Build PATCH /parent/notification-settings endpoint. Saves preferences: alert_type (stops/time), alert_value (number), child_id.

T24.3 — Build notification trigger in Notification Service. When a stop is completed (bus leaves a stop), check all parents whose child's stop is N stops away. If matches their preference, send FCM push. Same for ETA-based: check if ETA to parent's stop <= their configured minutes.

T24.4 — Build high-priority FCM channel for approaching alerts. Use Android notification channel with importance=HIGH. On iOS use critical alerts where possible. This ensures sound plays even on silent.

T24.5 — Build notification history screen (React Native). Scrollable list of all past notifications for all children. Shows type, child name, time, message.

M25 — Parent App — Trip History

Purpose: Parent can view past trips for their child, see attendance, and see which trips the child was absent.

Tasks — M25

T25.1 — Build trip history list (React Native). Filterable by month. Each row: date, shift, onboard time, drop time, driver name, attendance status.

T25.2 — Build trip detail view. Show route map with GPS trace (simplified — just key waypoints). Show child's onboard and drop times and locations.

M26 — Parent App — Settings

Purpose: Update child's stop, language preference, notification preferences, account details.

Tasks — M26

T26.1 — Build settings screen (React Native). Sections: Account (edit name, phone, email), Children (edit each child's stop/route), Notifications (per child preferences), Language (Telugu/Hindi/English).

T26.2 — Build stop update flow. Parent selects new stop from route's stop list. Calls PATCH /students/:id to update stop_id.

M27 — Backend — GPS Pipeline

Purpose: The core real-time engine. Receives GPS pings from driver app at 1-second intervals, validates them, stores them in TimescaleDB, broadcasts to Redis for WebSocket fanout to admin and parents.

Tech: Go service (high performance for concurrent GPS streams)

Acceptance Criteria:

- Handle up to 100 concurrent driver connections per school without degradation.
- Each GPS ping validated (lat/lng range, timestamp, trip_id must be active).
- Invalid pings logged and discarded, not stored.
- Valid pings stored to TimescaleDB gps_pings table.
- Valid pings published to Redis channel: trip:{tripId}:location.
- Speed calculated from consecutive pings if not provided by device.
- Heading calculated from last two positions if not provided by device.

Tasks — M27

T27.1 — Set up Go service with gorilla/websocket. WebSocket server that handles concurrent connections. Each connection authenticated via JWT in query param or header.

T27.2 — Build GPS ping handler. Receive JSON ping: { trip_id, lat, lng, speed, heading, accuracy, battery, timestamp }. Validate. Calculate speed/heading if missing. Write to TimescaleDB.

T27.3 — Build Redis publisher. After storing ping, publish to channel trip:{tripId}:location. Payload: { lat, lng, speed, heading, timestamp, tripId }.

T27.4 — Build batch sync endpoint. POST /location/batch in Node.js service (easier for JSON parsing). Accepts array of pings, inserts in bulk, re-runs geofence on each. Returns success count.

T27.5 — Set up TimescaleDB hypertable for gps_pings. Create table with chunk_time_interval of 1 hour. Add index on trip_id + ts. Set up compression policy for pings older than 7 days.

M28 — Backend — Geofence Engine

Purpose: For every GPS ping, check if the bus has entered the radius of the next expected stop. If yes, trigger stop arrival events.

Logic:

- Each trip tracks "current stop index" — which stop is next.
- On each GPS ping, calculate distance from current position to next stop using PostGIS ST_DWithin.
- If within radius: emit stop:arrived event to Redis.

- Trip Service listens and pushes attendance prompt to driver via WebSocket.
- Notification Service listens and checks if any parents need pre-alerts (2 stops before their stop).

Tasks — M28

T28.1 — Build geofence check in Location Service. After storing each GPS ping, run: SELECT id FROM stops WHERE ST_DWithin(geom, ST_Point($lng, $lat)::geography, $radius) AND id = $nextStopId. If match, publish stop:arrived:{tripId}:{stopId} event.

T28.2 — Build stop:arrived event consumer in Trip Service. On receiving event: update trip's current stop index. Push WebSocket message to driver's connection: { type: "STOP_ARRIVED", stopId, students: [...] }.

T28.3 — Build pre-alert check in Notification Service. On each stop:completed event, check all parents on this route. For any parent whose child's stop is exactly N stops ahead (matching their preference), queue a push notification via BullMQ.

T28.4 — Build off-route deviation check. Every 10 seconds (not every ping), check if bus position is more than 200m from the nearest point on the planned route polyline. If yes, create an alert.

M29 — Backend — Notification Engine

Purpose: Handles all push notifications using FCM (Android) and APNs (iOS). Uses BullMQ job queue so notifications are reliable and not blocking the main request flow.

Architecture:

- Any service can publish a notification job to Redis queue: notifications:queue
- Notification Worker picks up jobs, calls FCM/APNs APIs, logs result
- Failed notifications retried 3 times with exponential backoff

Tasks — M29

T29.1 — Set up BullMQ worker for notifications. Worker reads from Redis queue. Processes each job: look up user's FCM token, call Firebase Admin SDK sendMulticast. Log result.

T29.2 — Build FCM token registration endpoint. POST /notifications/register-token. Saves device FCM token for user. Called when app starts.

T29.3 — Build notification job publisher helper. Utility function used by all services: publishNotification(userId, title, body, data). Pushes to BullMQ queue.

T29.4 — Build high-priority channel for approaching alerts. Android: notification channel with importance=HIGH, bypass DND. iOS: use content-available + apns-priority=10.

T29.5 — Build notification logging. Every notification attempt logged to notifications table with sent_at and delivery status.

M30 — Backend — Offline Sync

Purpose: When driver's app reconnects after being offline, it posts all buffered data. Server must process it correctly preserving timestamps and running all side effects (geofence checks, attendance notifications).

Acceptance Criteria:

- Batch GPS endpoint processes pings in timestamp order.
- Geofence checks run on each historic ping (so if bus passed a stop while offline, stop:arrived still fires).
- Attendance events in batch also fire notifications to parents (slightly delayed, labeled as such).
- Duplicate pings (same trip_id + timestamp) are idempotent — ignored on second insert.

Tasks — M30

T30.1 — Build idempotency on GPS batch insert. Use INSERT ... ON CONFLICT (trip_id, ts) DO NOTHING in TimescaleDB. Prevents duplicates.

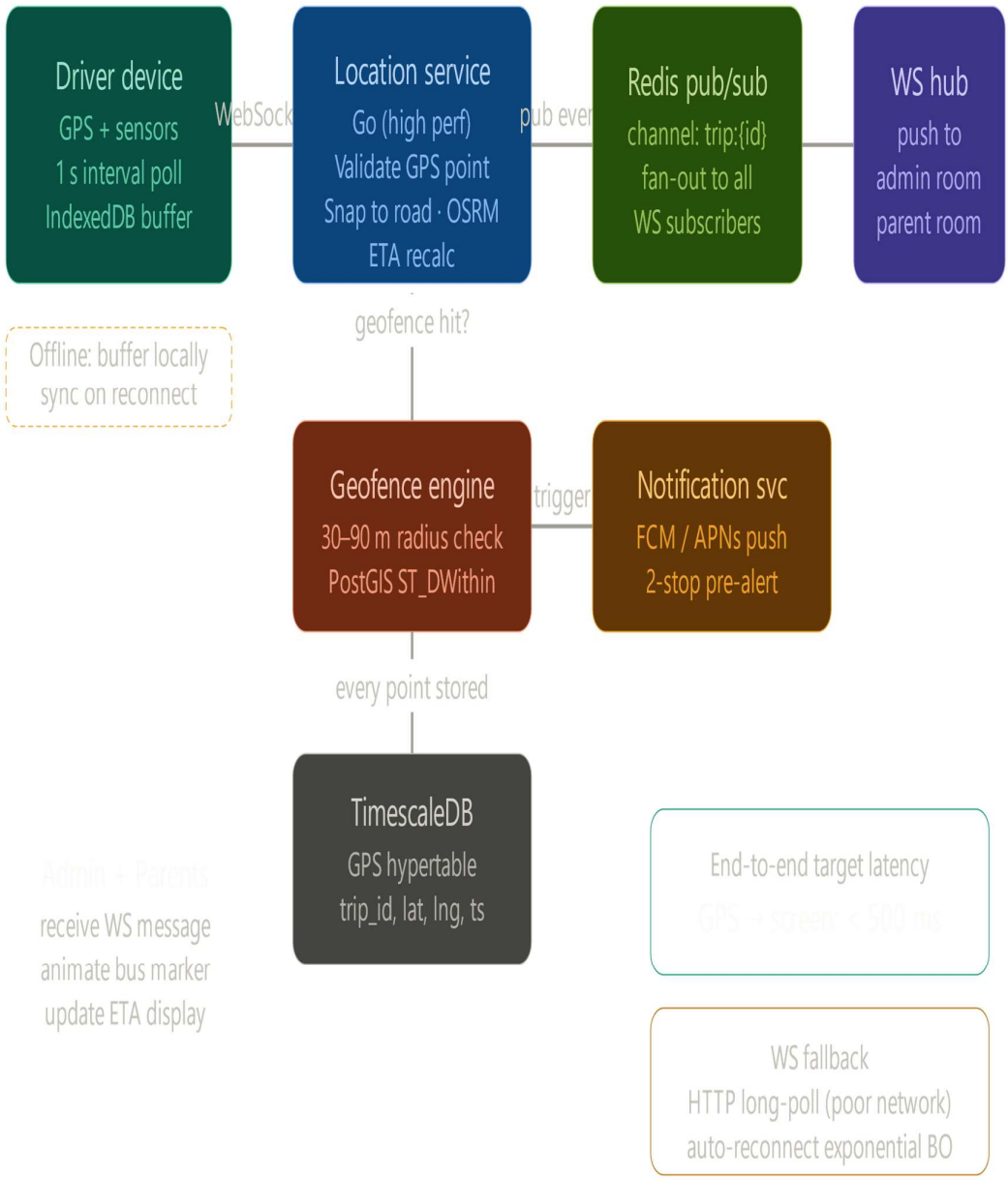
T30.2 — Build replay geofence on batch pings. After batch insert, run geofence check on each ping in order. Emit any stop:arrived events that would have fired in real time.

T30.3 — Build POST /attendance/batch endpoint. Accepts array of onboard/drop events with timestamps. Inserts in order. Fires parent notifications for each with a flag: { delayed: true } so notification body says "Aarav boarded the bus at 9:14 AM (updated)" rather than the current time.

Clients
Admin Panel
React Web App
Next.js / PWA
Driver App
React Native
iOS + Android
Parent App
React Native
iOS + Android
PWA Offline
Service Worker
IndexedDB cache
Layer 1
Edge
API Gateway
Caddy / Nginx
Rate limit · Auth JWT
WebSocket Hub
Socket.io / uWS
Sticky sessions · Redis pub
Push Gateway
FCM + APNs
Expo push proxy
Layer 2
Services
Auth Service
JWT · OTP
Role-based access
Node.js
Location Service
GPS ingest
Geofence engine
Go (high-perf)
Trip Service
Trip lifecycle
Attendance engine
Node.js
Notification Svc
Alert engine
Pre-alert scheduler
Node.js + BullMQ
Layer 3
Route Service
Route/stop CRUD
ETA calc · OSRM
Node.js
Analytics Svc
Delay · speed
Reports · export
Python / FastAPI
School/User Svc
School · driver
Student · parent
Node.js
Event Bus
Redis Streams
Async fanout
all services pub/sub
Data
PostgreSQL
Primary DB
Users · routes
trips · students
Redis
Live location
Session cache
Pub/sub · queues
TimescaleDB
GPS time-series
Trip history
Telemetry logs
S3 / R2
Photos · exports
CSV · trip logs
Cloudflare R2 free
Layer 4
Infra
Railway / Fly.io
Free tier hosting
Docker containers
Cloudflare
CDN · DDoS
R2 · free SSL
Supabase
Postgres free tier
Realtime · Auth
Upstash Redis
Serverless Redis
10k req/day free
Layer 5

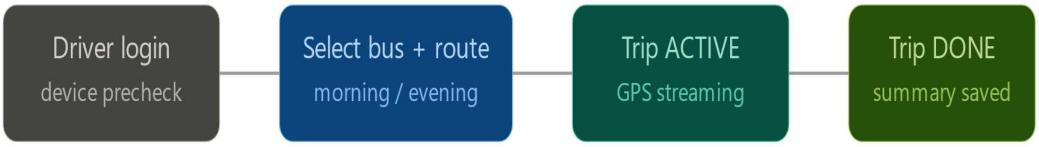
Real-time location pipeline

How driver GPS reaches admin + parents in under 500 ms

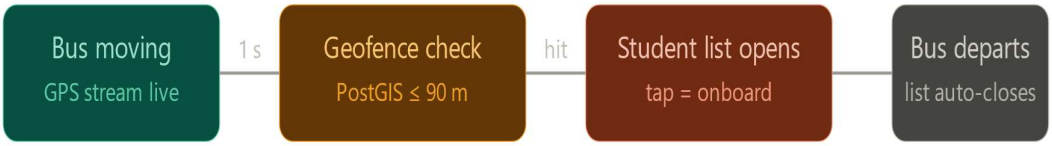


Database entity model

Trip state machine

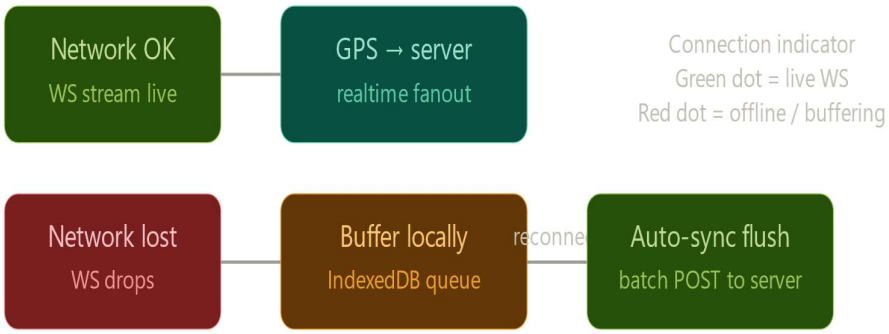


Automatic stop arrival flow



Notification trigger logic

Offline resilience (driver app)



REST API endpoints + permissions

Endpoint	Admin	Driver	Parent	Publ
— Auth —				
POST /auth/register-school	✓	—	—	✓
POST /auth/login	✓	✓	✓	✓
POST /auth/otp/send + verify	✓	✓	✓	✓
— Routes & Stops —				
GET /routes	✓	✓	✓	—
POST /routes (create)	✓	—	—	—
PATCH /routes/:id (edit)	✓	—	—	—
POST /routes/:id/stops	✓	—	—	—
— Trips —				
POST /trips/start	—	✓	—	—
POST /trips/:id/end	—	✓	—	—
GET /trips/active (admin monitor)	✓	—	—	—
GET /trips/:id (my trip)	✓	✓	✓ (child's)	—
— Location —				
POST /location/ping (GPS ingest)	—	✓	—	—
POST /location/batch (offline sync)	—	✓	—	—
GET /location/live/:tripId	✓		✓ (child's)	—
— Attendance —				
POST /attendance/onboard	—	✓	—	—
POST /attendance/drop	—	✓	—	—
GET /attendance?route=&date=	✓	—	✓ (child)	—
— Alerts & SOS —				
POST /alerts/sos	—	✓	—	—
POST /alerts/speeding (auto-trigger)	system			—
GET /alerts?trip=	✓	—	—	—
— WebSocket channels —				
ws: trip:{tripId} (location updates)	✓	✓ (send)	✓ (recv)	—
ws: school:{id}:fleet (admin fleet)	✓	—	—	—
ws: trip:{id}:attendance	✓	✓	—	—

eader. school_id is embedded in JWT — every query is auto-scoped to that school. No cross-school data leak possible.

BUILD ORDER (for Vibe Coding)

Use this sequence. Each phase is a complete working slice you can deploy and test.

Phase 1 — Foundation (Build first, everything depends on this) M27 (GPS pipeline backend) → M1 (Auth all apps) → M2 (School setup) → M6 (Bus management) → M4 (Driver management) → M5 (Student management) → M3 (Route/stop management)

Phase 2 — The Core Trip (The whole product value) M13 (Driver precheck) → M14 (Duty start) → M28 (Geofence engine) → M16 (Stop arrival + attendance) → M15 (Driver live dashboard) → M17 (SOS) → M18 (End trip) → M19 (Offline resilience)

Phase 3 — Admin Visibility M7 (Admin dashboard) → M8 (Live fleet map) → M9 (Trip monitoring) → M10 (Alerts)

Phase 4 — Parent Experience M29 (Notification engine) → M21 (Parent signup) → M22 (Parent dashboard) → M23 (Live tracking) → M24 (Notifications)

Phase 5 — Polish and Analytics M11 (Trip history + replay) → M12 (Analytics) → M20 (Driver settings) → M25 (Parent trip history) → M26 (Parent settings) → M30 (Offline sync polish)

VIBE CODING PROMPT STRATEGY

When you pass tasks to an AI coding tool, always include this context block at the top of every session:

```
Project: School Bus Tracking SaaS
Stack: Next.js 14 (admin), React Native + Expo (driver + parent apps),
Node.js + Express (API services), Go (location service), PostgreSQL +
Supabase, Redis + Upstash, TimescaleDB, Socket.io
Auth: JWT with school_id embedded — every query MUST be scoped to school_id
from req.user.school_id
Database: All tables have school_id column. Never query without WHERE
school_id = $schoolId
Real-time: WebSocket via Socket.io, Redis pub/sub for fanout
GPS storage: TimescaleDB hypertable gps_pings (trip_id, ts, lat, lng,
speed, heading)
Geofence: PostGIS ST_DWithin for stop radius checks
Notifications: FCM + APNs via Firebase Admin SDK, queued via BullMQ
Offline: Driver app buffers GPS in AsyncStorage, batch syncs on reconnect
Current task: [PASTE TASK HERE]
```

This ensures the AI model never loses context about the overall system and never generates code that breaks the school isolation, the real-time pipeline, or the offline resilience logic.

Total Modules: 30 Total Tasks: 107 Phases: 5 Estimated pilot-ready build time with focused vibe coding: 6–10 weeks solo